

# Scalable Multi-Tier Loan Application on AWS

Hands-on cloud deployment • Networking • Scaling • Monitoring

## PROJECT FOCUS

### Deploy • Scale • Monitor

EC2 | ALB | Auto Scaling | VPC | RDS | S3 | CloudWatch | SNS

#### COMPUTE

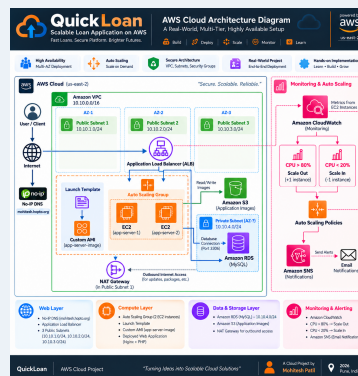
EC2 + Auto Scaling

#### NETWORK

VPC + ALB + NAT

#### OBSERVABILITY

CloudWatch + SNS



# Table of Contents

---

|           |                                                                           |           |
|-----------|---------------------------------------------------------------------------|-----------|
| <b>01</b> | <b>Project Overview</b><br>Purpose, scope and architecture summary        | <b>03</b> |
| <b>02</b> | <b>Technology Stack</b><br>Application, AWS services and deployment tools | <b>04</b> |
| <b>03</b> | <b>Architecture Diagram</b><br>End-to-end QuickLoan AWS architecture      | <b>05</b> |
| <b>04</b> | <b>VPC &amp; Networking</b><br>VPC, subnets, route tables and NAT Gateway | <b>06</b> |
| <b>05</b> | <b>Compute &amp; Traffic</b><br>EC2, AMI, Launch Template and ALB         | <b>07</b> |
| <b>06</b> | <b>Auto Scaling</b><br>Auto Scaling Group and scaling policies            | <b>08</b> |
| <b>07</b> | <b>Monitoring</b><br>CloudWatch CPU alarms and scaling signals            | <b>09</b> |
| <b>08</b> | <b>Data &amp; Storage</b><br>RDS MySQL and Amazon S3                      | <b>10</b> |
| <b>09</b> | <b>Application</b><br>QuickLoan application deployment                    | <b>11</b> |
| <b>10</b> | <b>Alerting</b><br>Amazon SNS notification setup                          | <b>12</b> |
| <b>11</b> | <b>AWS Service Map</b><br>Services used across the solution               | <b>13</b> |
| <b>12</b> | <b>Project Takeaways</b><br>Skills and implementation summary             | <b>14</b> |

## QUICKLOAN • AWS CLOUD PROJECT

Scalable multi-tier loan application deployed on Amazon Web Services

A Cloud Project by Mohitesh Patil

# Project Overview & Architecture

QuickLoan • Scalable Multi-Tier AWS Cloud Deployment

## QuickLoan

QuickLoan is a cloud-hosted loan application web platform built with a PHP backend and HTML/CSS/JavaScript frontend. The project demonstrates a structured AWS infrastructure setup covering networking, compute, storage, database, load balancing, auto scaling and monitoring.

The application is deployed inside a custom Virtual Private Cloud (VPC) with segmented public and private subnets. Traffic is distributed through an Application Load Balancer (ALB) to an Auto Scaling Group of EC2 instances, while the application connects to a managed Amazon RDS for MySQL database. Static application images are stored in Amazon S3.

### APPLICATION

#### PHP + Nginx

Web application and server-side logic

### COMPUTE

#### EC2 + Auto Scaling

Scalable application instances

### DATA

#### RDS MySQL + S3

Database and application assets

### MONITORING

#### CloudWatch + SNS

CPU alarms and notifications

### PROJECT REGION

US East (Ohio) • us-east-2

### AUTHOR

Mohitesh Patil

# Technology Stack

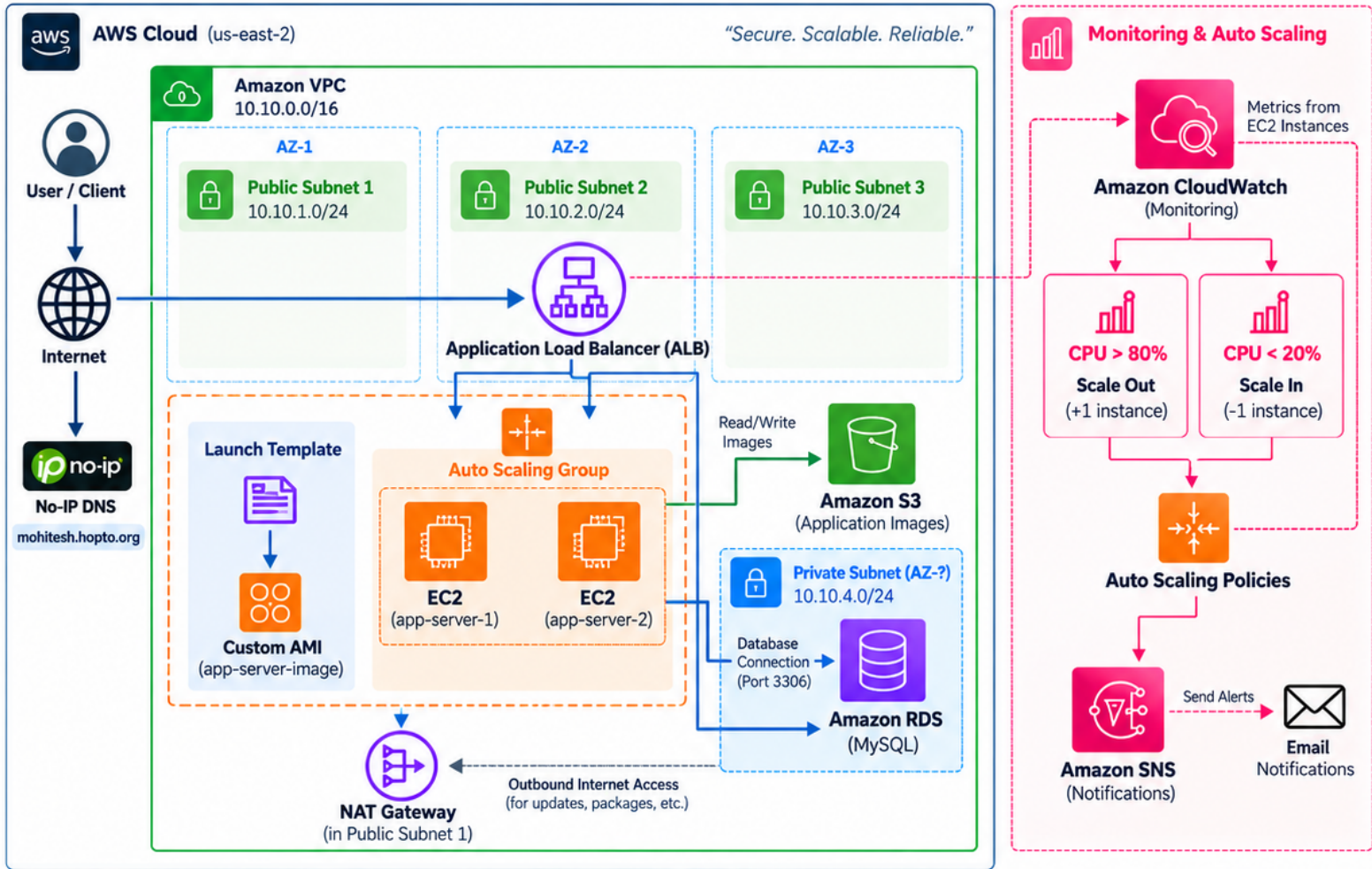
QuickLoan • Application, AWS infrastructure and deployment components

| LAYER          | TECHNOLOGY / SERVICE                              | PURPOSE                                               |
|----------------|---------------------------------------------------|-------------------------------------------------------|
| Frontend       | HTML, CSS, JavaScript                             | User interface of the QuickLoan web application       |
| Backend        | PHP 8.2, PHP-FPM                                  | Server-side application logic                         |
| Web Server     | Nginx                                             | HTTP request handling and application routing         |
| Database       | Amazon RDS – MySQL                                | Persistent storage for application data               |
| Object Storage | Amazon S3                                         | Storage for application image assets                  |
| Networking     | VPC, Subnets, Route Tables, NAT Gateway           | Structured and isolated cloud network                 |
| Compute        | Amazon EC2                                        | Application server instances                          |
| Scaling        | Auto Scaling Group + Launch Template + Custom AMI | Horizontal scaling and repeatable instance deployment |
| Load Balancing | Application Load Balancer (ALB)                   | Traffic distribution across EC2 instances             |
| Monitoring     | Amazon CloudWatch                                 | CPU metrics and scaling alarms                        |
| Alerting       | Amazon SNS                                        | Notification delivery for monitoring events           |
| DNS / Domain   | No-IP (mohitesh.hopto.org)                        | Dynamic DNS access for the application                |
| File Transfer  | WinSCP                                            | Application file transfer to EC2 during deployment    |

## PROJECT STACK

AWS • Linux • Nginx • PHP • MySQL • CloudWatch • SNS

A Cloud Project by Mohitesh Patil



**Web Layer**

- No-IP DNS (mohitesh.hopto.org)
- Application Load Balancer
- 3 Public Subnets (10.10.1.0/24, 10.10.2.0/24, 10.10.3.0/24)

**Compute Layer**

- Auto Scaling Group (2 EC2 instances)
- Launch Template
- Custom AMI (app-server-image)
- Deployed Web Application (Nginx + PHP)

**Data & Storage Layer**

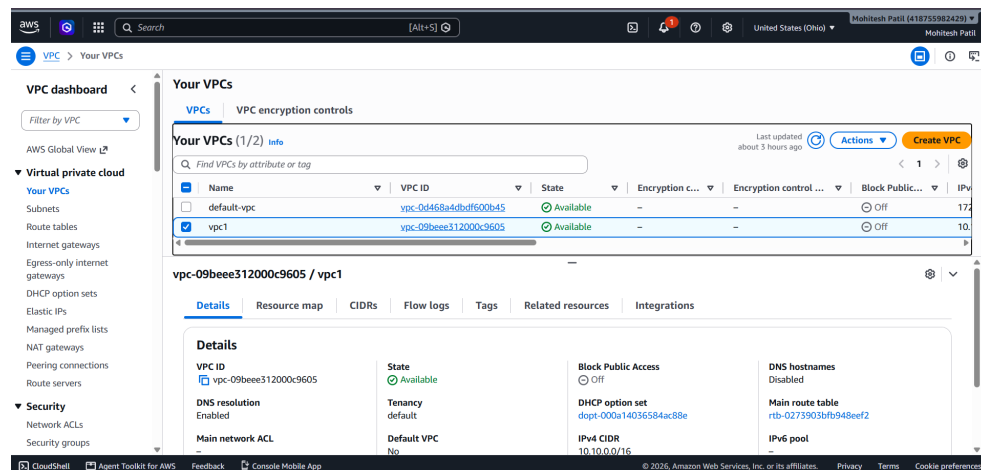
- Amazon RDS (MySQL) – 10.10.4.0/24
- Amazon S3 (Application Images)
- NAT Gateway for outbound access

**Monitoring & Alerting**

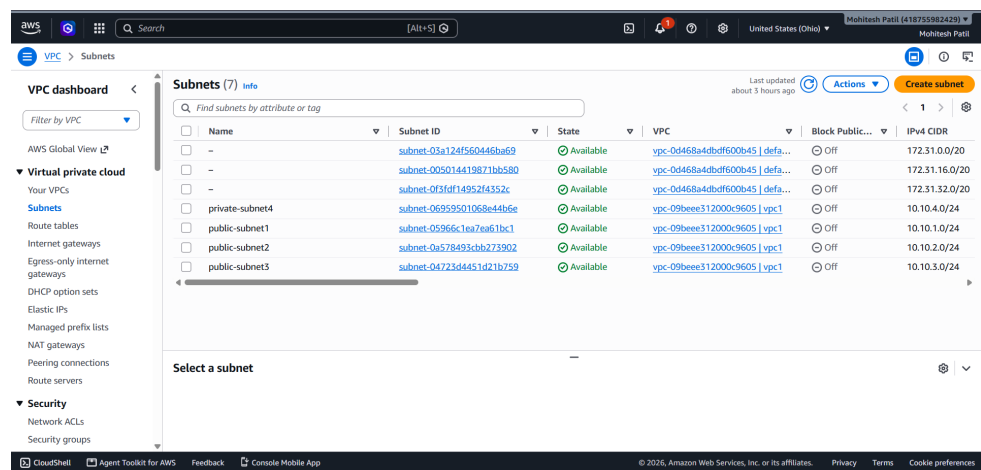
- Amazon CloudWatch
- CPU > 80% → Scale Out
- CPU < 20% → Scale In
- Amazon SNS (Email Notifications)

# A structured VPC foundation.

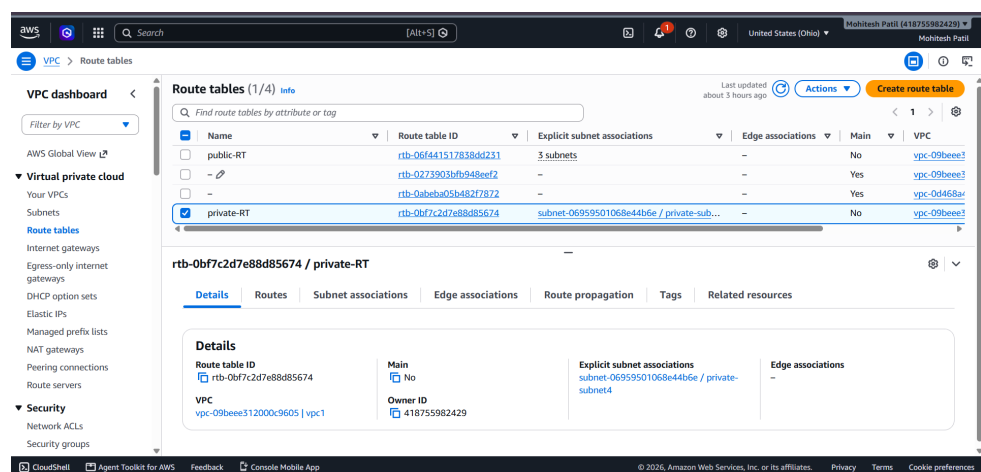
The project uses a custom VPC with separate subnet ranges and routing components. The screenshots below document the actual VPC configuration used during deployment.



Custom VPC configuration



Subnet layout

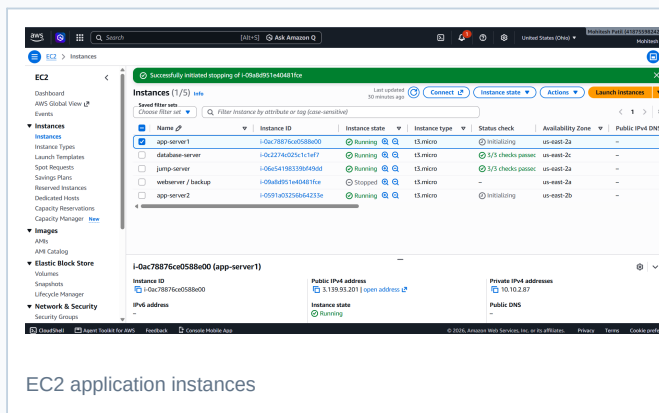


Route tables

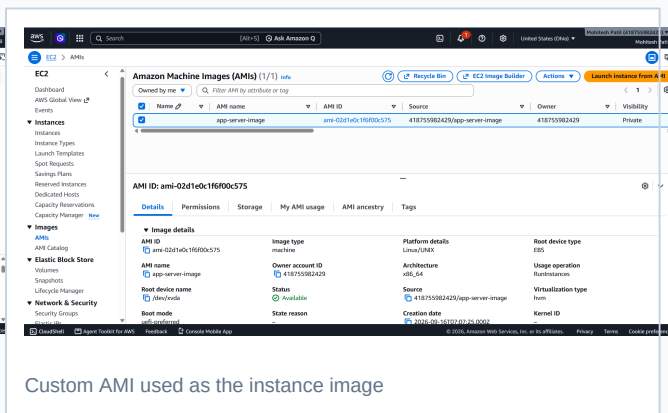


# Scale the application layer.

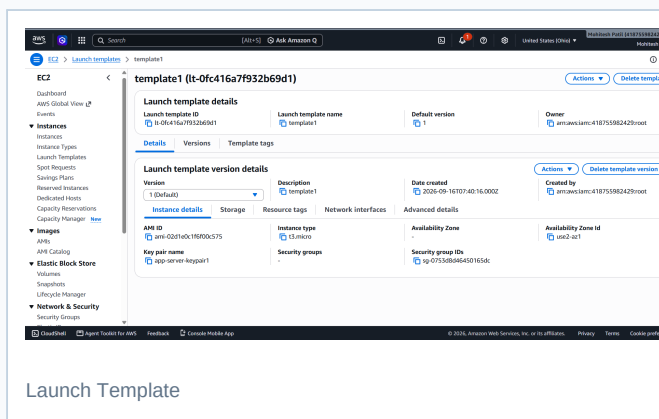
The compute layer uses EC2, a custom AMI, a launch template, an Auto Scaling Group and an Application Load Balancer. This gives the application a repeatable instance configuration and a load-balanced entry point.



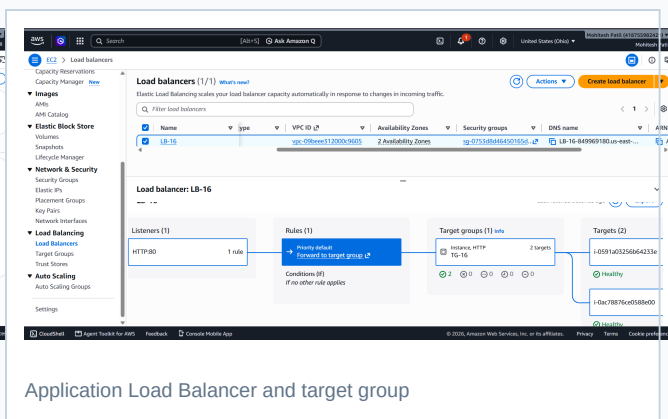
EC2 application instances



Custom AMI used as the instance image



Launch Template



Application Load Balancer and target group

# Capacity responds to CPU pressure.

The Auto Scaling configuration uses a launch template and CPU-based policies. The project evidence shows the group configured with desired capacity of 2 and scaling limits of 1 to 5 instances.

**Auto Scaling groups (1/1)** info Last updated less than a minute ago

Search your Auto Scaling groups

| Name       | Status              | Launch template/configuration | Instances | Instance health | Desired capacity |
|------------|---------------------|-------------------------------|-----------|-----------------|------------------|
| ASG-16SEPT | At desired capacity | template1   Version Default   | 2         | 2/2 Healthy     | 2                |

**Auto Scaling group: ASG-16SEPT**

**ASG-16SEPT Capacity overview**

arn:aws:autoscaling:us-east-2:418755982429:autoScalingGroup:fb37173-a982-40d6-bf10-052b6aa00924:autoScalingGroupName/ASG-16SEPT

| Desired capacity | Scaling limits | Desired capacity type       | Status              |
|------------------|----------------|-----------------------------|---------------------|
| 2                | 1 - 5          | Units (number of instances) | At desired capacity |

**Date created**  
Wed Sep 16 2026 13:15:09 GMT+0530 (India Standard Time)

Auto Scaling Group

**server-creation-policy**

Policy type: Simple scaling

Enabled or disabled: Enabled

Execute policy when alarm for **cpu > 80 %** breaches the alarm threshold: CPUUtilization > 80 for 1 out of 1 periods of 300 seconds for the metric dimensions.  
AutoScalingGroupName = ASG-16SEPT

Take the action: Add 1 capacity units

And then wait: 300 seconds before allowing another scaling activity

**server-removal-policy**

Policy type: Simple scaling

Enabled or disabled: Enabled

Execute policy when alarm for **cpu < 20 %** breaches the alarm threshold: CPUUtilization > 20 for 1 out of 1 periods of 300 seconds for the metric dimensions.  
AutoScalingGroupName = ASG-16SEPT

Take the action: Remove 1 capacity units

And then wait: 300 seconds before allowing another scaling activity

**Predictive scaling policies (0)** info

Evaluation period

Scaling policies

## SCALING LOGIC

### SCALE OUT

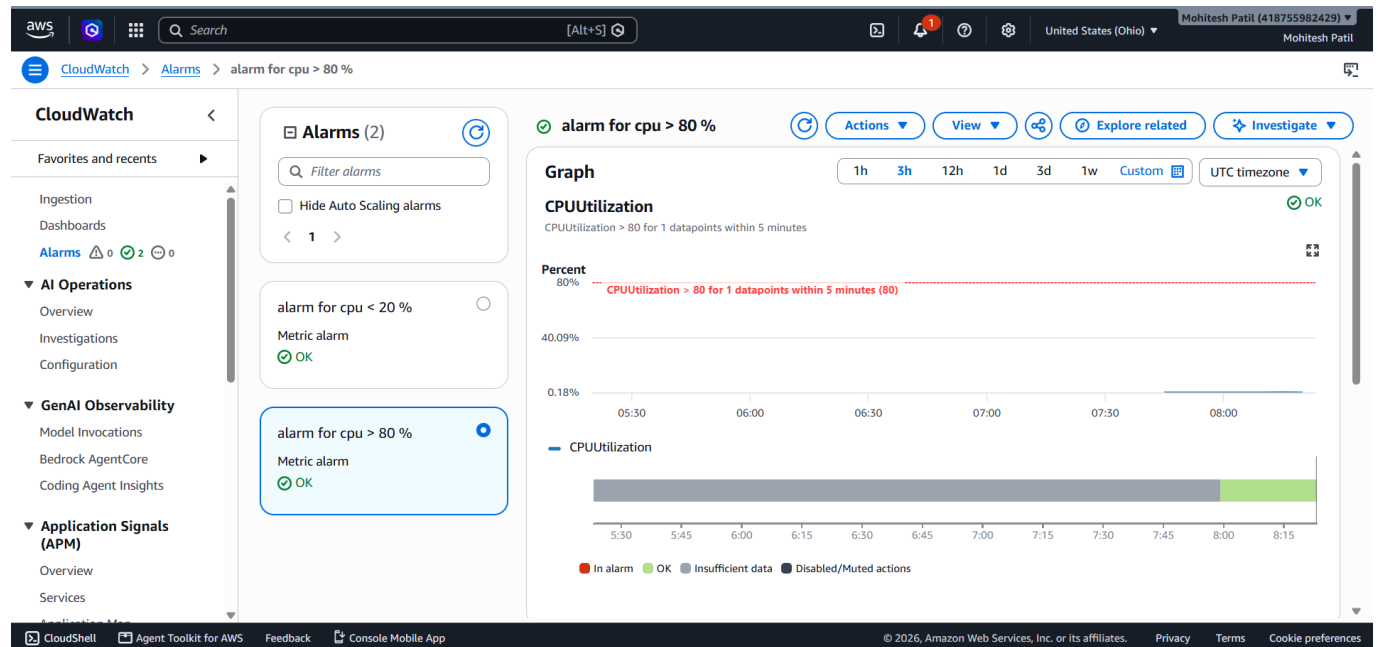
CPU > 80% alarm provides the high-utilization signal for increasing capacity.

### SCALE IN

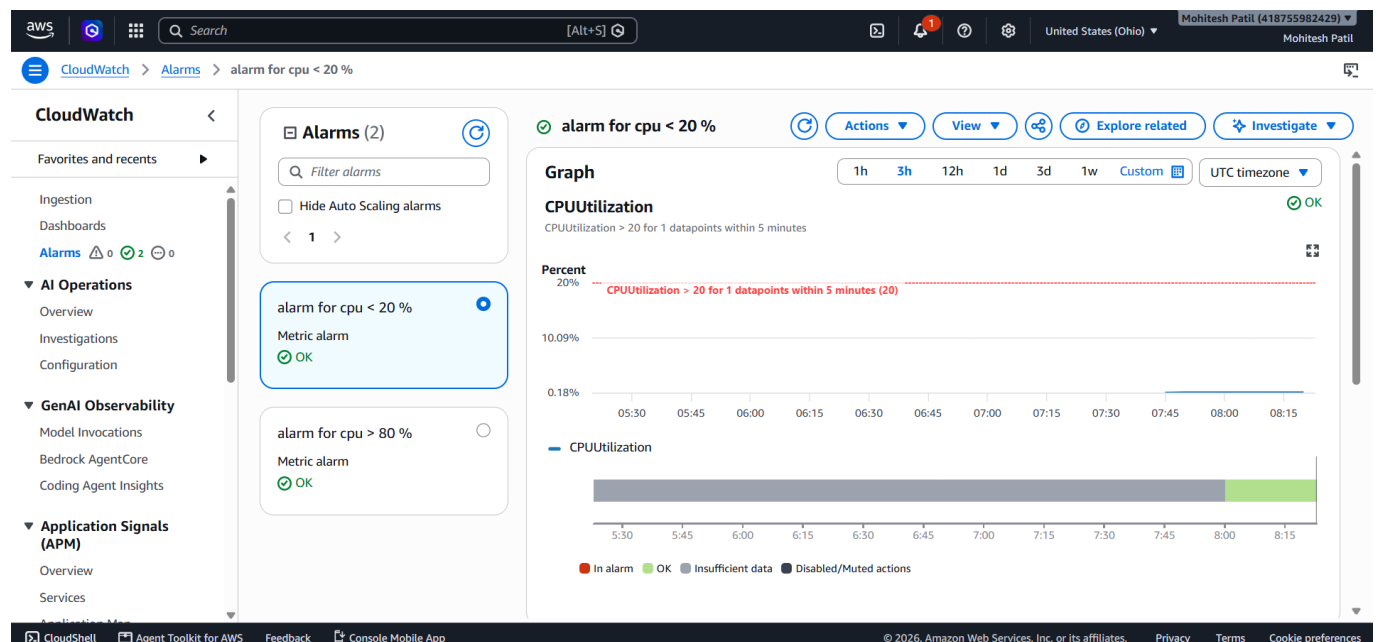
CPU < 20% alarm provides the low-utilization signal for reducing capacity.

# CloudWatch signals the scaling layer.

These two CloudWatch alarm views document the CPU thresholds configured for the project. Both alarms are shown in an OK state in the supplied screenshots.



CPU > 80% — high-utilization alarm

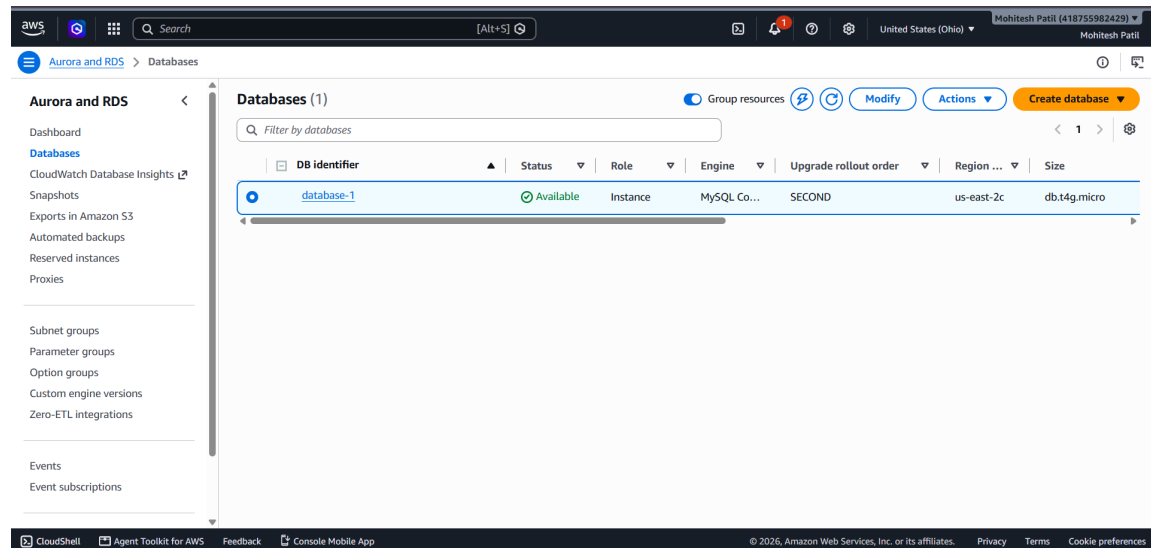


CPU < 20% — low-utilization alarm

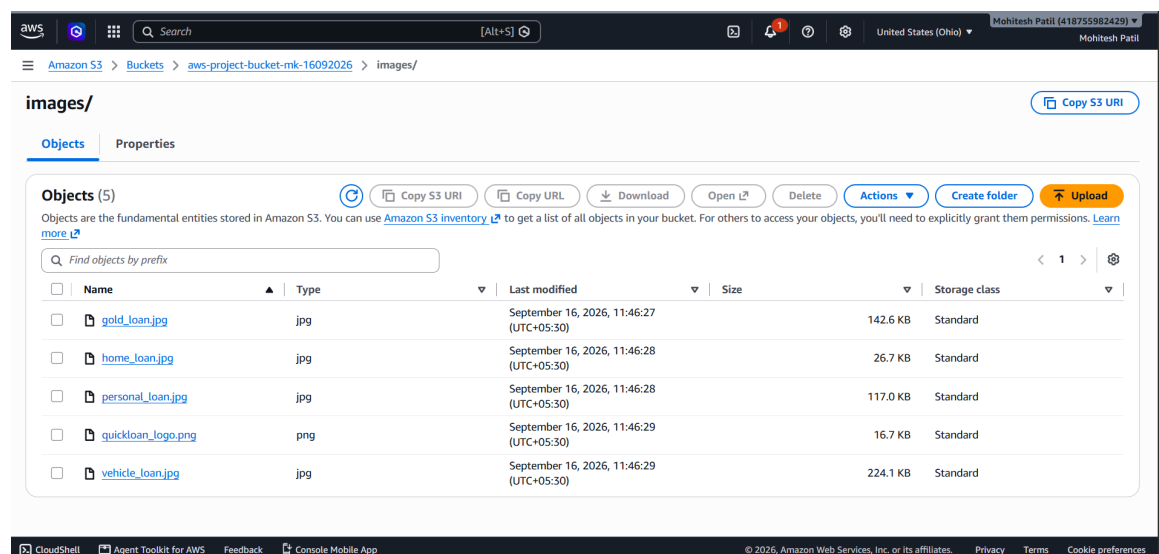
The alarms are the signals; the Auto Scaling policies determine the corresponding capacity action.

# Managed database. Object storage.

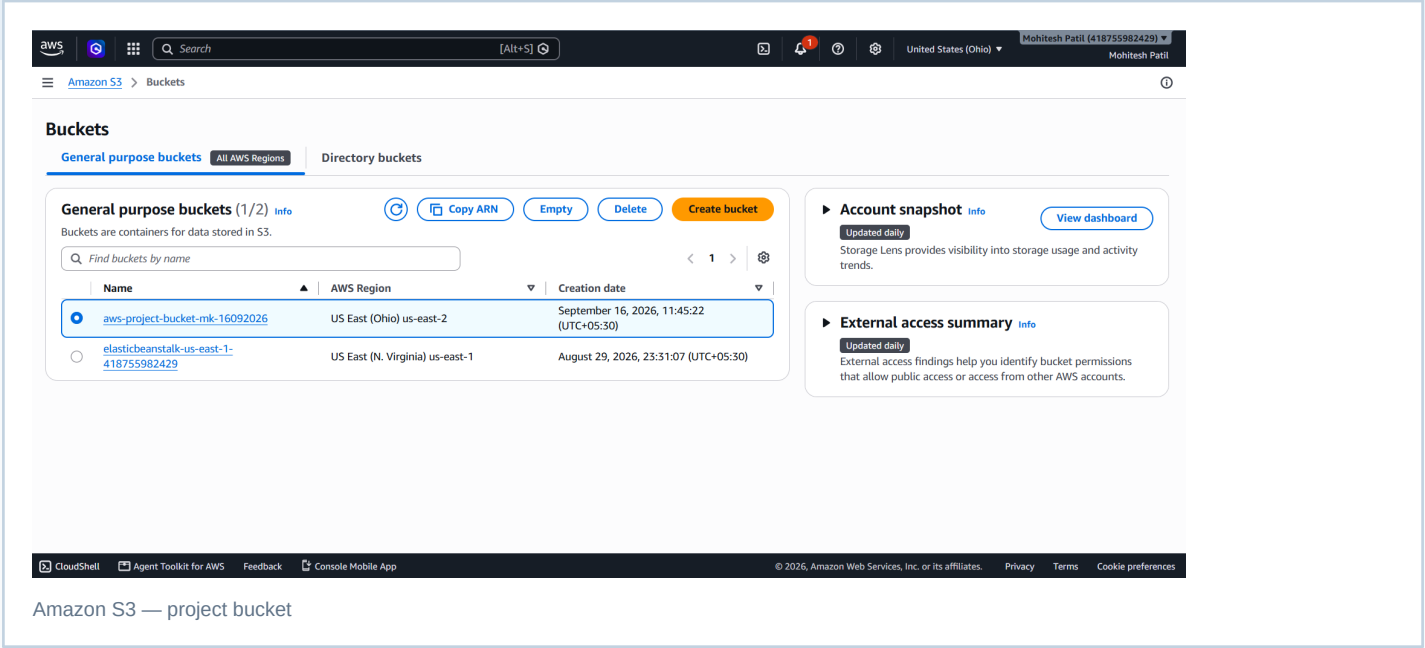
Application data and static assets are separated into dedicated AWS services. RDS provides the relational database layer while S3 stores image assets.



Amazon RDS — MySQL database

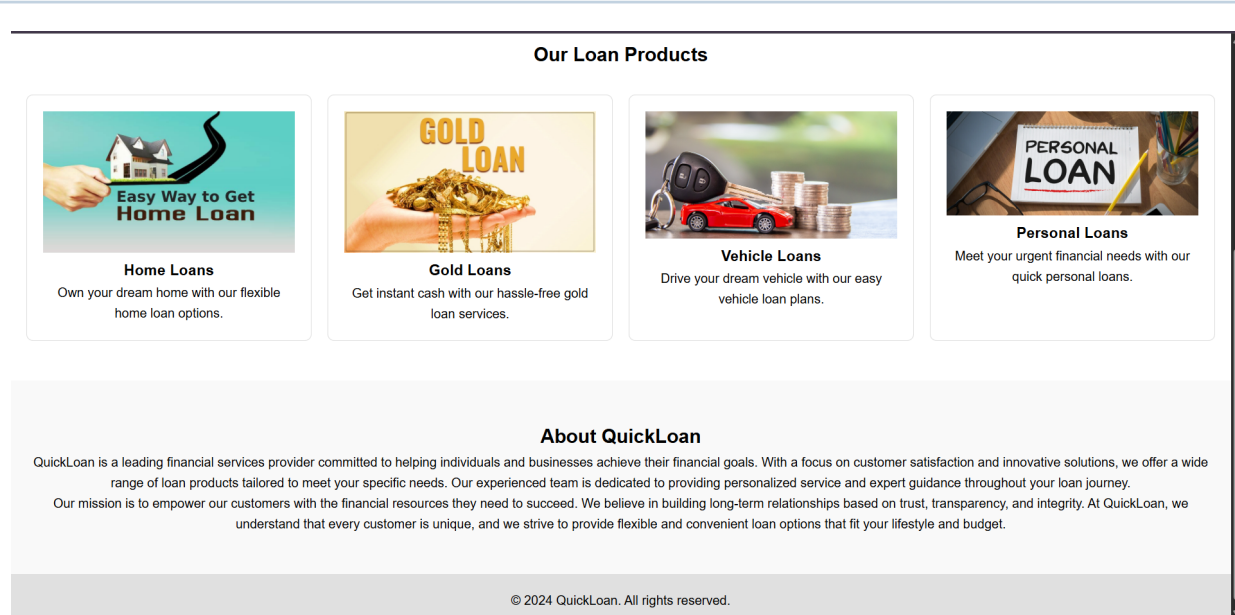


Amazon S3 — application images

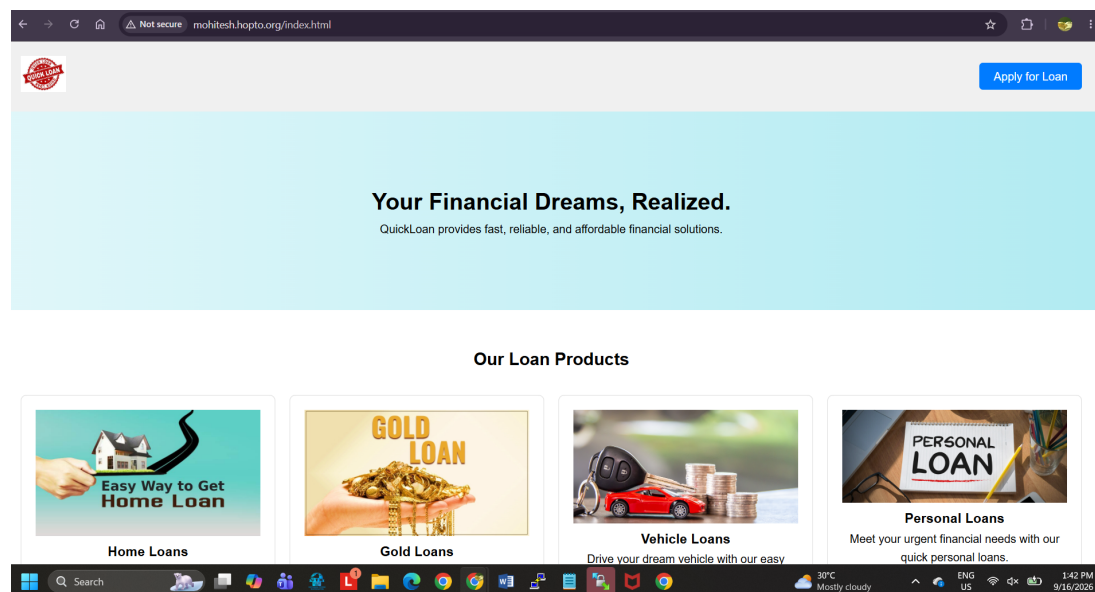


# The cloud stack serves a working application.

The deployment was validated through the live QuickLoan application and its loan product interface.



Loan products interface



Application homepage

my.noip.com/dns/records?jump\_to\_zone=hopto.org

English

mohiteshpatil5@gmail.com

Dashboard

DDNS & Remote Access

Managed DNS

Zones

DNS Records

Backup DNS

No-IP Client

Domains

Email

Server Monitoring

SSL Certificates

Team Login

Add Priority Support

Records

My No-IP > Records

Create Hostname

Free Dynamic DNS: 1/1

hopto.org

Upgrade to Enhanced Dynamic DNS

Free Dynamic DNS accounts are limited to 1 Hostname, currently there is 1 Hostname in your account.

Upgrade Now

hopto.org 1 record

No-IP Owned

Add Record

| Name               | Type  | Content                                     | TTL | Last Update           |
|--------------------|-------|---------------------------------------------|-----|-----------------------|
| mohitesh.hopto.org | CNAME | LB-16-849969180.us-east-2.elb.amazonaws.com | 60  | Sep 16, 2026 00:50:06 |

my@3a519bc99 2026-09-14T15:02:51Z web03

DNS configuration used for the project hostname

# Notifications complete the monitoring loop.

Amazon SNS is configured as the notification component associated with the monitoring setup. The supplied console screenshots document the project topic and subscription state.

The screenshot shows the Amazon SNS console interface. On the left, the navigation menu includes 'Amazon SNS', 'Dashboard', 'Topics', 'Subscriptions', and 'Mobile'. The main content area is titled 'Topics (1)' and displays a table with one topic: 'topic-aws-project-16sept'. The table columns are 'Name', 'Type', and 'ARN'. The topic is of type 'Standard' and its ARN is 'arn:aws:sns:us-east-2:418755982429:topic-aws-proj...'. Above the table, there are buttons for 'Edit', 'Delete', 'Publish message', and 'Create topic'. A blue banner at the top of the main content area announces a new feature: 'Amazon SNS now supports High Throughput FIFO topics. Learn more'.

Amazon SNS topic

The screenshot shows the Amazon SNS console interface, specifically the 'Dashboard' view. The left navigation menu is the same as in the previous screenshot. The main content area is titled 'Dashboard' and features a 'Resources for us-east-2' section with three metrics: 'Topics' (1), 'Platform applications' (0), and 'Subscriptions' (1). Below this, there is an 'Overview of Amazon SNS' section with a sub-header 'Application-to-application (A2A)'. The text describes Amazon SNS as a managed messaging service that decouples publishers from subscribers. A diagram illustrates the A2A architecture: a 'Publisher' sends messages to 'Amazon SNS' (Fully managed Pub/Sub messaging and event-driven), which then sends them to an 'SNS topic'. The topic can be subscribed to by 'Subscribers' (AWS Lambda, Amazon SQS, Amazon Kinesis Data Firehose, HTTP/HTTPS, Email) or a 'Dead-letter queue' (if an endpoint is unavailable, messages can be held in a dead-letter queue for analysis or reprocessing). The diagram also shows 'Message filtering and fanout'.

Amazon SNS console view

# The stack, at a glance.

Services documented through the project configuration and supplied screenshots.

## EC2

Compute instances

## ALB

Traffic distribution

## ASG

Elastic capacity

## VPC

Network isolation

## RDS

MySQL database

## S3

Object storage

## CloudWatch

Monitoring

## SNS

Notifications

## AMI

Reusable image

## Launch Template

Instance blueprint

## NAT Gateway

Outbound private access

## No-IP

Project hostname

PROJECT STACK / AWS • LINUX • NGINX • PHP • MYSQL • CLOUDWATCH

# What this project demonstrates.

01

**Cloud deployment**

Deployed a working loan application using multiple AWS infrastructure services.

02

**Networking**

Configured a custom VPC with public/private subnet structure, route tables and NAT Gateway.

03

**Scalability**

Implemented an Application Load Balancer with Auto Scaling, launch template and CPU-based policies.

04

**Managed services**

Used RDS MySQL for relational data and S3 for application image assets.

05

**Monitoring**

Configured CloudWatch CPU alarms and SNS-based notification components.

06

**Hands-on operations**

Worked directly with EC2, AMIs, Nginx, DNS configuration and AWS console resources.

**RESUME-READY SUMMARY**

Deployed a scalable multi-tier PHP loan application on AWS using EC2, Application Load Balancer, Auto Scaling, RDS MySQL, S3, VPC, CloudWatch and SNS. Configured networking, reusable instance templates and CPU-based scaling policies to support a structured cloud deployment.

## BUILD • DEPLOY • MONITOR • SCALE